

1. 单例模式

单例模式 (Singleton Pattern) 是一种创建型设计模式, 它确保一个类仅有一个实例, 并提供一个全局访问点来获取该实例。这种模式在很多情况下都非常有用, 尤其是在那些需要一个对象来协调动作并确保对象一致性的场景中。

1.1 实现思路

1. 确保用户无法自己创建对象(构造函数私有化, 删除拷贝构造函数)
2. 让类自己负责管理这唯一的一个对象(使用静态成员变量)
3.
 - 普通的成员属于对象, 而静态成员变量不属于对象。
 - 不能在构造函数中定义和初始化, 需要在类的外部单独定义和初始化。
 - 静态成员变量和全局变量一样存放在全局区, 可以把静态成员变量理解成是被限制在类中去使用的全局变量。
4. 提供一个安全的全局访问点, 即提供一个公开的接口, 让用户可以使用这唯一的一个实例 (静态成员函数)
5.
 - 静态成员函数没有this指针, 没有const属性, 但是可以重载 (静态与非静态成员函数名字相同且参数相同不构成重载)
 - 访问方式:
 - 类名::静态成员函数(实参表); // 推荐
 - 对象名.静态成员函数(实参表); // 和上面方法本质一样
 - 在静态成员函数中只能访问静态成员, 不能访问普通成员, 在普通的成员函数中, 既可以访问静态的成员也可以访问普通的成员。

不同的语言, 不同的程序员, 可能有多种实现方式, 总体上可以分为两种: 饿汉式和懒汉式

1.1.1 饿汉式

饿汉式单例模式(预先创建): (无论用或不用, 程序启动即创建)

- 唯一的实例在类加载的时候创建, 预先初始化(不管有没有用到, 都创建这个实例)
- 在类加载的时候创建, 调用的时候反应速度快
- 在类加载的时候创建, 有可能永远也用不到这个实例(资源浪费)

静态成员变量是在类加载时就创建并初始化的, 因此在程序开始前就已经创建了单例对象

```
// 单例模式: 饿汉式
#include <iostream>
using namespace std;

class Singleton
{
public:
    // 3) 通过静态成员函数获取单例对象
    // 如果不返回引用, 则会有临时变量的产生, 调用拷贝构造函数, 而拷贝构造函数又被禁用了
    static Singleton& getInstance(void)
    {
        return s_instance;
    }
};
```

```

void print(void)
{
    cout << m_data << endl;
}
private:
//1) 私有化构造函数
Singleton(int data = 0) :m_data(data)
{
    cout << "创建单例对象" << endl;
}
Singleton(const Singleton&) = delete;

//2) 通过静态成员变量维护唯一的对象
//静态成员变量是在类加载时就创建并初始化的，因此在程序开始前就已经创建了单例对象
static Singleton s_instance;
private:
    int m_data;
};
//std::cout << "111" << std::endl;

Singleton Singleton::s_instance(12345);
int main()
{
    //Singleton s1;
    //Singleton *ps = new Singleton(); //error 构造函数被私有化了。
    cout << "main开始执行" << endl;
    //如果不定义为引用，则会调用构造函数生成新对象，但是构造函数被私有化了。
    Singleton &s1 = Singleton::getInstance();
    Singleton &s2 = Singleton::getInstance();
    //定义一个引用绑定到s1，不会产生新的对象
    //Singleton &s2 = s1;
    //Singleton& s2(s1);

    cout << "&s1 = " << &s1 << endl;
    cout << "&s2 = " << &s2 << endl;

    s1.print();
    s2.print();
    return 0;
}

```

- 注意：
 - **问题1**：为什么 class Singleton 可以包含 static Singleton s_instance 静态成员？

静态成员的生命周期：静态成员变量与类本身相关联，而不是与类的任何特定对象相关联（静态成员不属于类）。静态成员变量的生命周期是整个程序的执行期间。这意味着静态成员变量在程序开始运行时被创建，并在程序结束时被销毁。这个特性使得静态成员变量非常适合用于实现单例模式，因为单例模式要求有一个全局唯一的对象实例，该实例在程序的生命周期内一直存在。

初始化时机：在类定义外部，我们可以对静态成员变量进行初始化。这通常是在类定义之后的某个地方，或者在包含类定义的源文件的顶部。这个初始化只执行一次，且发生在程序启动时。因此，对于饿汉式单例模式，我们在类外部初始化静态成员变量 s_instance，这样当程序开始运行时，单例对象就已经被创建并准备好了。

访问权限：静态成员变量可以通过类名直接访问，而不需要创建类的对象。这使得静态成员变量成为实现单例模式的理想选择，因为单例模式的设计目标就是允许全局访问唯一的对象实例，而不需要（也不允许）创建该类的多个实例。

- ○ **问题2:** `class Singleton` 可以包含 `Singleton s_instance` 非静态成员吗?

不可以, 因为 `class Singleton` 还未创建出来, 不确定其大小, 但是可以包含 `Singleton *s_instance` 类型指针变量, 因为指针的大小是确定的。

- ○ **问题3:** 为什么在 `Singleton &s1 = Singleton::getInstance();` 要在 `s1` 前加 `&`

因为加 `&` 表示 `s1` 是这个单例对象的引用, 不产生新的对象; 而不加 `&` 表示创建一个新的 `s1` 对象, 用这个单例对象来初始化, 但是构造函数被私有化了, 类外无法访问, 拷贝构造函数被禁用了。

- ○ **问题4:** 饿汉式单例模式中, 单例对象具体是在什么时候创建的?

在饿汉式单例模式中, 单例对象是在程序启动时创建的, 具体来说, 是在全局静态变量的初始化阶段。这个初始化过程发生在程序的静态存储区分配和初始化阶段, 通常是在 `main` 函数执行之前 (新词将该单例初始化代码放在 `main` 函数后也会比 `main` 函数先执行)。

对于饿汉式单例模式, 当你定义了一个静态成员变量并给它一个初始值, 如 `static Singleton s_instance(12345);`, 这个变量会在程序启动时自动被初始化。这个初始化过程是由编译器在编译时确定的, 不需要任何额外的代码或函数调用。

下面是一个更详细的解释:

当编译器遇到类定义中的静态成员变量时, 它会记住这个变量需要在程序启动时初始化。

在链接阶段, 编译器和链接器会合作确定所有全局和静态变量的初始化顺序。对于静态成员变量, 它们通常按照它们在代码中的声明顺序进行初始化。

当程序开始执行时, 在 `main` 函数之前, 全局对象和静态对象 (包括静态成员变量) 的构造函数会被调用, 进行初始化。

对于饿汉式单例模式中的 `static Singleton s_instance(12345);`, 当程序开始执行时, `s_instance` 的构造函数会被调用, 使用提供的参数 `12345` 进行初始化。

一旦 `s_instance` 被初始化, 它就存在于程序的整个生命周期中, 可以通过 `Singleton::getInstance()` 方法安全地访问。

由于饿汉式单例模式的初始化是在程序启动时完成的意味着即使单例对象在程序的某些部分中并未使用, 它仍然会占用内存空间。

相比之下, 懒汉式单例模式将对象的创建推迟到第一次需要它的时候, 这可以节省内存, 但在多线程环境下需要额外的同步机制来确保线程安全。

1.1.2 懒汉式

- 懒汉式单例模式(延时创建); (用时再创建,不用即销毁)
- ○ 唯一的实例在第一次调用获取实例接口的时候创建,延时初始化
- ○ 不用到就永远不会创建
- ○ 避免了饿汉式那种在没有用到的情况下也会创建实例的问题,资源利用率高
- ○ 在使用的时候创建,第一次调用的时候反应速度没有饿汉式快


```

//单例模式:懒汉式
#include<iostream>
using namespace std;

class Singleton
{
public:
    //3)通过静态成员函数获取单例对象
    static Singleton& getInstance(void)
    {
        if (s_instance == NULL)
        {
            s_instance = new Singleton(54321);
        }
        ++s_count;
        return *s_instance;
    }
    //单例可以被多个人同时使用,应该是最后有个使用者负责回收
    void release(void)
    {
        if (--s_count == 0)
        {
            delete s_instance;
            s_instance = NULL;
        }
    }
    void print(void)
    {
        cout << m_data << endl;
    }
private:
    //1)私有化构造函数
    Singleton(int data = 0) :m_data(data)
    {
        cout << "创建单例对象" << endl;
    }
    Singleton(const Singleton&) = delete;
    ~Singleton(void)
    {
        cout << "单例对象被销毁了" << endl;
    }

    //2)通过静态成员变量维护唯一的对象
    static Singleton* s_instance;
    //计数,用于记录使用单例对象的人数
    static int s_count;
private:
    int m_data;
};
Singleton* Singleton::s_instance(NULL);
int Singleton::s_count = 0;

int main()
{
    //Singleton s1;
    //Singleton *ps = new Singleton();
    cout << "main开始执行" << endl;
}

```



```

Singleton& s1 = Singleton::getInstance();//1
Singleton& s2 = Singleton::getInstance();//2
cout << "s1 = " << &s1 << endl;
cout << "s2 = " << &s2 << endl;

s1.print();
s1.release();//1
s2.print();
s2.release();//0
return 0;
}

```

- 问题：

- ○ **问题1：**为什么这个实例中单例对象要定义为 `static Singleton* s_instance;` 而不是 `static Singleton s_instance;`

在懒汉式单例模式的实现中，单例对象通常定义为指向单例类型的静态指针（`static Singleton* s_instance;`），而不是静态对象（`static Singleton s_instance;`），主要有以下几个原因：

延迟初始化：懒汉式单例模式的核心思想是将单例对象的创建延迟到第一次真正需要它的时候。如果使用静态对象，那么单例对象会在程序启动时就被创建，这违背了懒汉式单例的初衷。而使用静态指针，我们可以控制单例对象的创建时机，确保在第一次调用 `getInstance` 方法时才进行创建。

动态内存分配：使用指针允许我们动态地分配内存给单例对象。这意味着我们可以利用 `new` 操作符在堆上创建单例对象，并通过指针来管理它。这种方式提供了更大的灵活性，特别是在处理大型对象或需要更多控制对象生命周期的情况下。

易于销毁：通过指针，我们可以更容易地销毁单例对象。当不再需要单例对象时，我们可以使用 `delete` 操作符释放其占用的内存。如果使用静态对象，则对象的生命周期将与程序的执行期相同，无法手动销毁。虽然对于单例模式来说，通常不需要显式销毁单例对象，但在某些特殊情况下（如单元测试或资源清理），手动销毁可能是有益的。

- ○ **问题2：**饿汉式单例对象的创建时机？

在这个例子中，`getInstance()` 方法是获取单例对象的入口。当第一次调用这个方法时，会检查 `instance` 是否为 `null`。如果是 `null`，则创建一个新的 `Singleton` 对象，并将其赋值给 `instance`。之后，每次调用 `getInstance()` 方法都会直接返回这个已经创建好的对象，而不会再创建新的对象。

1.1.3 二者的区别

1. 对象创建时机：

懒汉式单例：在第一次使用时才创建对象实例。也就是说，当第一次调用获取单例对象的方法时，才会进行对象的实例化。这种方式可以节省系统资源，因为只有在真正需要该对象时才会进行实例化。

饿汉式单例：在类加载时就创建对象实例。无论是否使用，单例对象在类被加载时就已经存在。从资源利用效率的角度来看，饿汉式单例稍差于懒汉式，因为无论是否使用，对象都会被创建。但从速度和反应时间角度来看，饿汉式单例稍好于懒汉式，因为它避免了首次使用时的同步或条件检查开销。

2. 资源初始化和性能：

懒汉式单例：在实例化时，必须处理好多线程同时首次引用该类时访问限制的问题，特别是在单例类作为资源控制器时，资源初始化可能会耗费时间。

饿汉式单例：对象在类加载时就已存在，无需担心多线程访问或资源初始化的问题，因此性能上可能稍优。

1.2 单例的特点和用途

1. **全局唯一实例**：无论尝试多少次实例化这个类，它都只会返回一个对象实例。
2. **减少资源消耗**：当一个类只需要一个对象实例时，使用单例模式可以避免创建多个对象，从而减少资源消耗。
3. **提供访问控制**：单例模式可以确保对资源的访问是同步的，从而避免多线程环境下的并发问题。